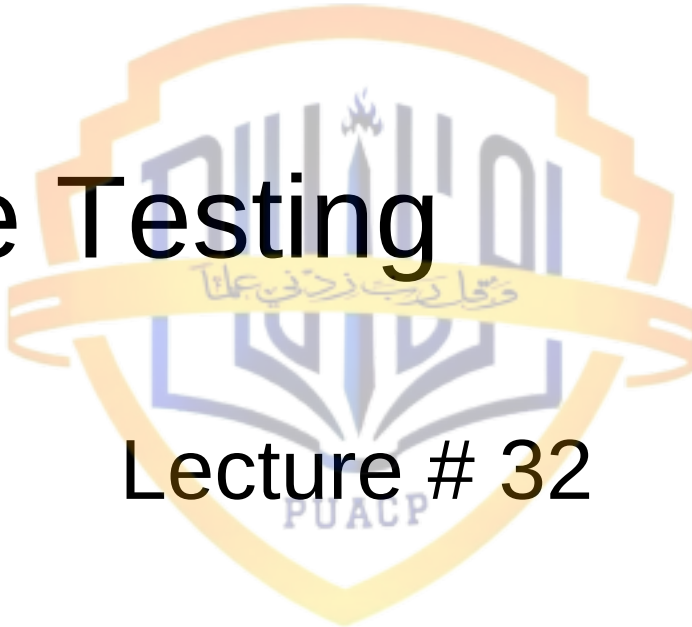




Software Testing



Lecture # 32

Recap and Today's Lecture

- We were discussing automated software testing and test tools, when the last class ended
- This is our last lecture on software testing in this course on software quality assurance
- Today we will talk about
 - Test cases
 - Software testers
 - Testing completion criteria

Test Cases



Test Cases - 1

- A test case describes how each test is to be conducted and also describes **input/output details**
- Development of test cases assist in keeping track of what is tested, when it is tested, and the outcome of the test
- If a defect is found in testing, a documented test case makes it easy for the developer to re-create the problem so that proper analysis can be done to fix it

Test Cases - 2

- Selecting test cases is the single most important task that software testers do. Improper selection can result in testing too much, testing too little, or testing the wrong things
- Intelligently weighing the risks and reducing the infinite possibilities to a manageable effective set is where the magic is

Test Cases - 3

- When designing and running your test cases, always run the test-to-pass cases first. It is important to see if the software fundamentally works before you throw the kitchen sink at it. You may be surprised how many bugs you find just by using the software normally

Design of Test Cases - 1

- There are no standards or simple rules for designing test cases
- Test cases should be designed for testing each function, with expected results
- In order to determine what will break the system, the tester has to be creative and **want to break** the system

Design of Test Cases - 2

- The test cases should be repeatable, i.e., when a test case is rerun, it should give the same results each time, except in real-time environment
- Test cases should be organized and developed around system requirements and for key functions of the system

Design of Test Cases - 3

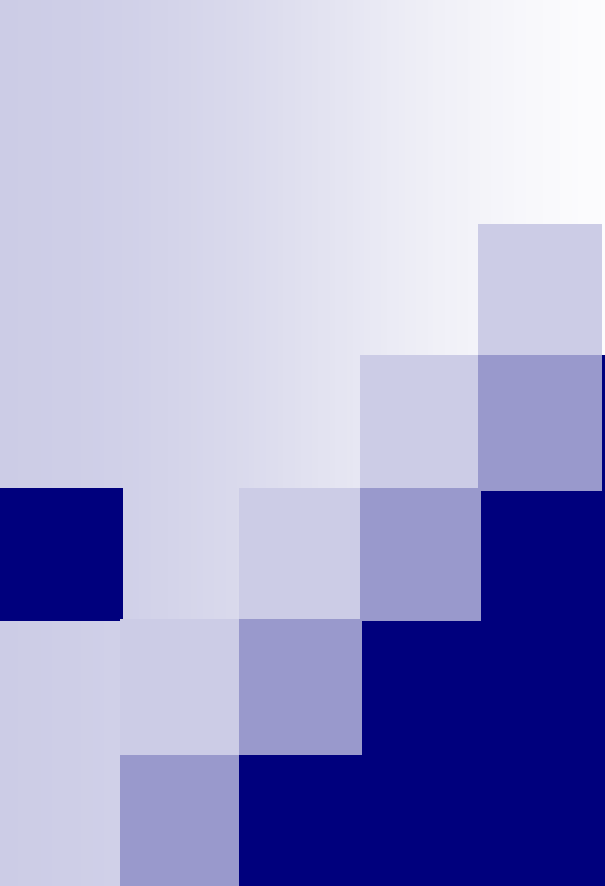
- The test design emphasizes
 - The type of test needed to test the requirements
 - How a test will run
 - How all requirements will be addressed by the tests
- Good test cases catch errors
- Develop test cases that cover as many related functions as possible

Design of Test Cases - 4

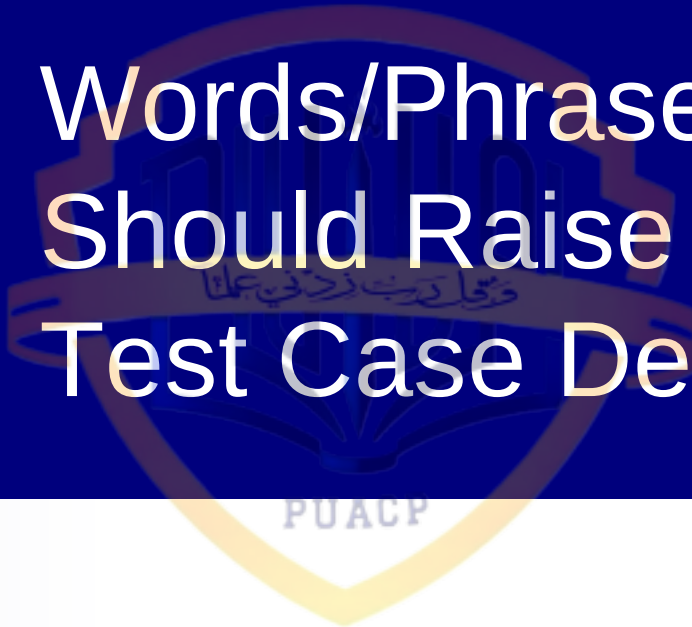
- A test case contains
 - Test case identifier
 - The name of the person responsible for executing the test
 - The date the test was executed and the version of the system tested
 - Description of the functions to be tested and a list of actions to be carried out

Design of Test Cases - 5

- Input values
- Output values (expected results). In most cases, the success of a test depends on obtaining the predefined expected results
- Actual results (output received when the test was executed). Indicate whether or not errors were detected, and if so, complete a “problem report”



Words/Phrases That Should Raise an Alarm in Test Case Design



Design of Test Cases - 6

- Always, every, all, none, never
 - If you see words such as these that denote something as certain or absolute, make sure that it is, indeed, certain. Think of cases that violate them
- Certainly, therefore, clearly, obviously, evidently
 - These words tend to persuade you into accepting something as given. Don't fall into the trap

Design of Test Cases - 7

- Some, sometimes, often, usually, ordinarily, customarily, most, mostly
 - These words are too vague. It's impossible to test a feature that operates "sometimes"
- Etc., And so forth, and so on, such as
 - Lists that finish with words such as these aren't testable. More explanation is needed

Design of Test Cases - 8

- Good, fast, cheap, efficient, small, stable
 - These are unquantifiable terms. They are not testable
- Handled, processed, rejected, skipped, eliminated
 - These terms can handle large amounts of functionality that need to be specified
- If...then...(but missing else)
 - Look for statements that have “if...then” clauses but don’t have a matching “else.” Ask yourself what will happen if the “if” doesn’t happen

Characteristics of a Test Case

Repeatable

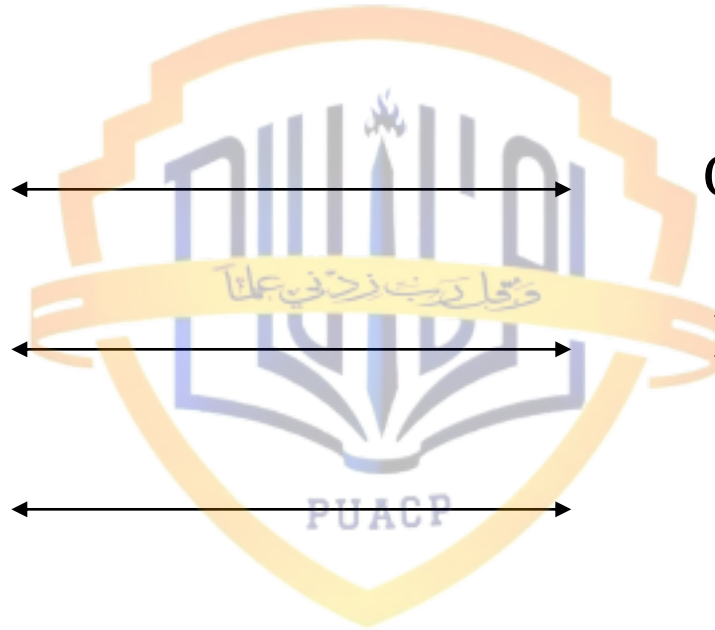
Consistent

Destructive

Detects Discrepancy

Manageable

Results Can Be Traced



Types of Test Cases

- Functional Test Cases
- Computation and Boundary Analysis Test Cases
- Cause-Effect and Logic Cases
- Security Test Cases

Test Case Development Considerations

- All valid system input must be accepted
- All invalid system input must be rejected with clear error messages
- All functions – including generation of reports – should be tested
- Any reports generated by the system should be tested and verified
- Any interfacing systems must be invoked and verified that the data is passed to and from the interfacing system

Bad Test Cases - 1

- The error density in software test cases is often higher than the error density in the software product
- Sometimes more than 15% of total number of test cases created can have errors themselves
- 50% of repair effort is applied towards test cases themselves

Bad Test Cases - 2

- Running test cases that are flawed or in error is of no value in terms of quality
- There is an implied need for greater rigor in building and validating test cases and test libraries

Common Problems in Test Cases - 1

- Testing limits and ranges where those included in the test-case are themselves incorrect
- Testing for numeric values where the test data contains wrong data
- Test cases derived from specifications or user requirements which contained undetected errors that were accidentally passed on to the test cases
- Accidental redundancy of test cases or test libraries (30%)

Common Problems in Test Cases - 2

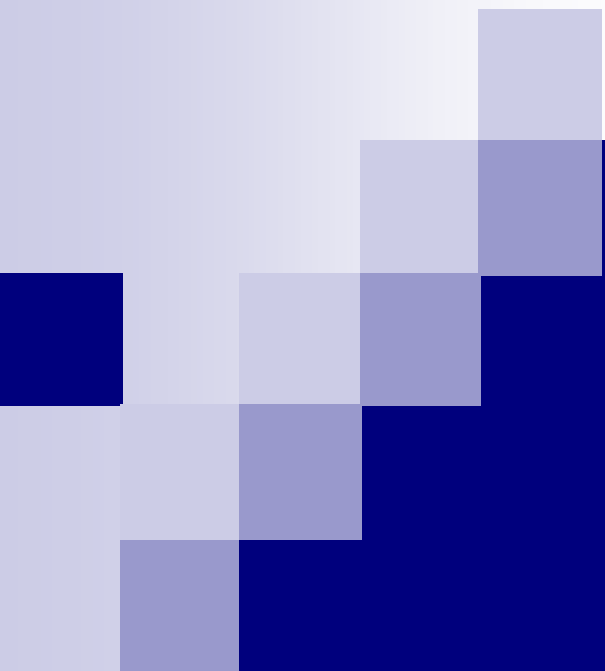
- Gaps, or portions of the application for which no test case exist (70%)
 - Result of gaps in requirements
- Formal inspections on test cases are very effective in eliminating bad test cases

Interesting Observations

- Test cases created by the programmers themselves have the lowest probability of being duplicated, but the highest probability of containing errors or bugs
- Test cases created later by test or quality assurance personnel have the greatest probability of redundancy but lower probability of containing errors

Research in Test Case Generation





Software Testers



Who Should Do Testing?

- The developers themselves
- Professional test personnel
- Professional quality assurance personnel
- Some combination of the above three

Some Observations About Testers - 1

- The defect removal efficiency of black-box testing is higher when performed by test personnel or by quality assurance personnel rather than developers themselves
- Black-box testing performed by clients (Beta and acceptance testing) varies widely, but efficiency rises with the numbers of clients involved

Some Observations About Testers - 2

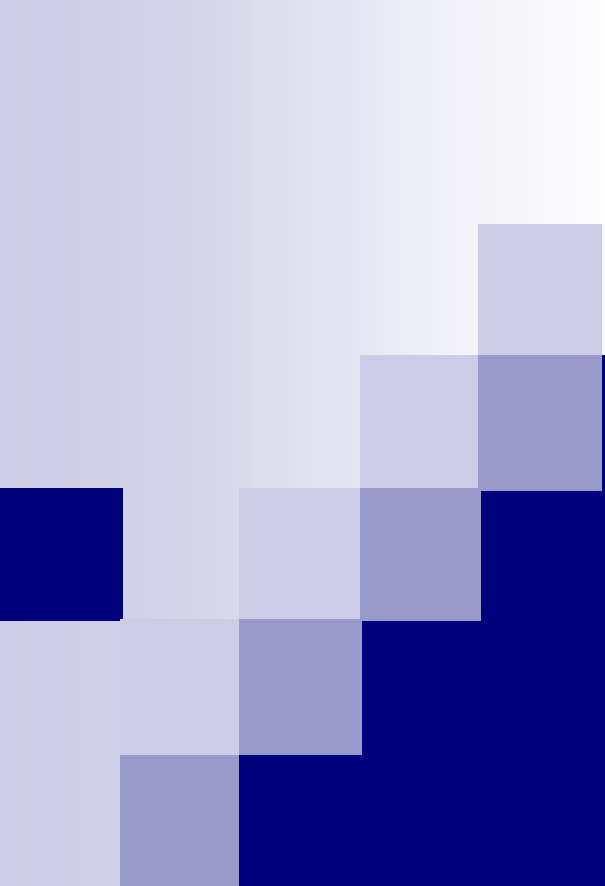
- For usability problems, testing by clients themselves outranks all other forms of testing
- The defect removal efficiency of white-box subroutine and unit testing stages is highest when performed by developers themselves

Some Observations About Testers - 3

- The defect removal efficiency of specialized kinds of white-box testing such as Year 2000 testing or viral protection testing is highest when performed by professional test personnel rather than by the developers themselves

Distribution of Testers

- On average
 - Developers do 31% of testing
 - Professional test personnel do 31% of testing
 - Quality assurance personnel do 18% of testing
 - Clients do 20% of testing



When to Stop Testing?

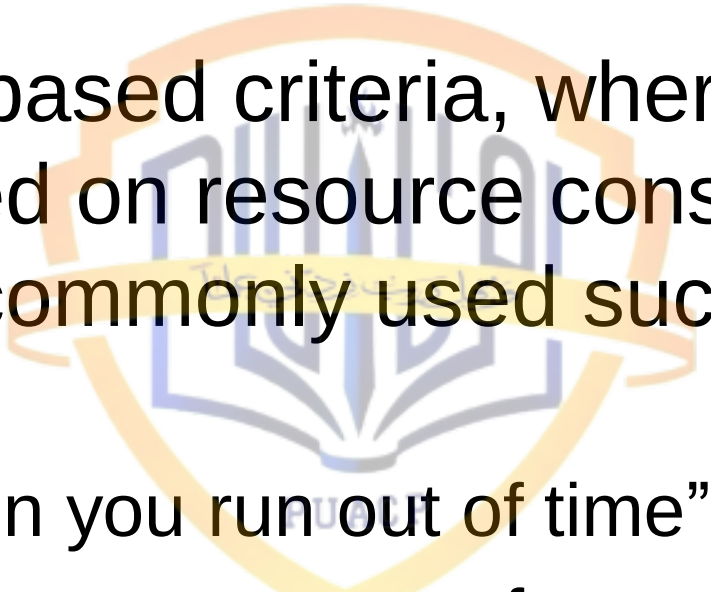


Criteria for Completion of Testing

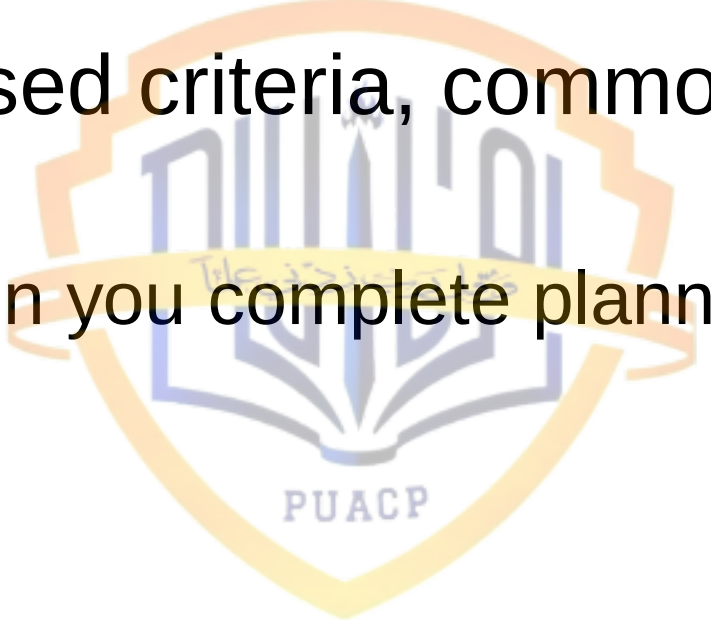
- You're never done testing, the burden simply shifts from you to your customer
- You're done testing when you run out of time or you run out of money
- There is no definitive answer

When to Stop Testing?

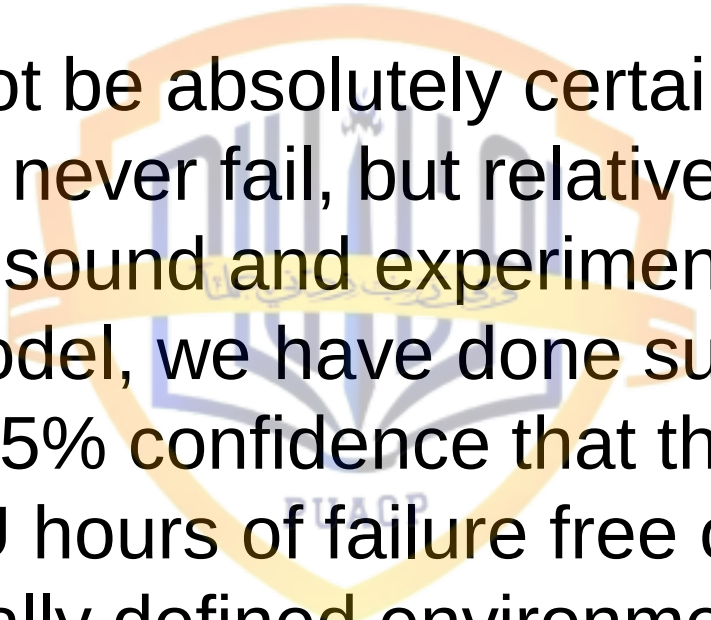
- On a small or local scale, we can ask: “When to stop testing for a specific test activity?”
- On a global scale, we can ask: “When to stop all the major test activities?” because the testing phase is usually the last major development phase before product release, this question is equivalent to: “When to stop testing and release the product?”

- 
- 
- Resource-based criteria, where decision is made based on resource consumptions. The most commonly used such stopping criteria are
 - “Stop when you run out of time”
 - “Stop when you run out of money”

- 
- Such criteria are irresponsible, as far as product quality is concerned, although they may be employed if product schedule or cost are the dominant concerns for the product in question

- 
- 
- Activity-based criteria, commonly in the form:
 - “Stop when you complete planned test activities”

- 
- This criterion implicitly assumes the effectiveness of the test activities in ensuring the quality of the software product. However, this assumption could be questionable without strong historical evidence based on actual data from the project concerned

- 
- 
- No we cannot be absolutely certain that the software will never fail, but relative to a theoretically sound and experimentally validated statistical model, we have done sufficient testing to say with 95% confidence that the probability of 1000 CPU hours of failure free operation in a probabilistically defined environment is at least 0.995

- Musa and Ackerman

Summary

- Today we had discussed test cases, responsibilities and characteristics of software testers, and completion criteria
- Recap entire series of software testing lectures

References

- Inroads to Software Quality by Alka Jarvis and Vern Crandall (Chapter 6.4 and 6.6)
- Software Engineering: A Practitioner's Approach by Roger S. Pressman (Chapter 18.1.4)
- Jeff Tian (Chapter 6.4)
- Software Quality: Analysis and Guidelines for Success by Capers Jones